

Rapport — Application Flutter de Ventes Privées

Table des matières

1. Architecture choisie	1
1.1. Vue d'ensemble	1
1.2. Backend (Python / Flask)	2
1.3. Frontend (Flutter / Dart)	2
2. Difficultés rencontrées	3
2.1. 1. Communication réseau entre le serveur et l'émulateur	3
2.2. 2. Gestion des états asynchrones	3
2.3. 3. Persistance des favoris	3
2.4. 4. Filtrage côté serveur	3
3. Limites de sécurité identifiées	3
4. Conclusion	4
5. Auteur	4

1. Architecture choisie

1.1. Vue d'ensemble

L'application suit une architecture **client-serveur** classique, composée de deux parties indépendantes communiquant via une API REST en HTTP :

- Un **backend Python (Flask)** qui expose des endpoints et lit les données depuis des fichiers JSON.
- Un **frontend Flutter (Dart)** qui consomme l'API et gère l'état local avec le pattern **Provider**.

Couche	Technologie
Backend	Python 3 + Flask 3.1
Frontend	Flutter 3.13+ (Dart 3)
État	Provider (ChangeNotifier)
Persistance session	shared_preferences
HTTP	paquet http
Stockage données	Fichiers JSON

1.2. Backend (Python / Flask)

Le serveur est un fichier unique (`server.py`) qui :

1. Charge les fichiers `users.json` et `ventes.json` en mémoire au démarrage.
2. Expose trois endpoints REST :
 - `POST /api/login` — authentification par email + mot de passe
 - `POST /api/register` — création de compte
 - `GET /api/produits` — liste des produits (avec filtres `?search=` et `?categorie=`)
 - `GET /api/produits/<id>` — détail d'un produit
3. Renvoie les réponses au format JSON avec gestion des erreurs (401, 404, 400).

Le mot de passe est volontairement **exclu** de la réponse de connexion pour éviter son exposition côté client.

1.3. Frontend (Flutter / Dart)

L'application Flutter est organisée en trois couches :

- `models/` — contient le modèle `User` avec les champs `id`, `email`, `nom`, `prenom`.
- `services/` — contient trois providers :
 - `AuthProvider` — gère la connexion, la déconnexion, et la persistance via `SharedPreferences`.
 - `FavoritesProvider` — gère les favoris localement (liste d'IDs persistés dans `SharedPreferences`).
 - `api_service.dart` — constante contenant l'URL de base de l'API.
- `screens/` — quatre écrans :
 - `login_screen.dart` — formulaire de connexion avec validation et gestion d'erreur.
 - `register_screen.dart` — formulaire d'inscription (bonus).
 - `product_screen.dart` — grille de produits avec barre de recherche, filtres par catégorie, et bouton favoris.
 - `product_detail_screen.dart` — affichage détaillé d'un produit.

1.3.1. Gestion d'état avec Provider

L'application utilise `MultiProvider` à la racine (`main.dart`) pour injecter `AuthProvider` et `FavoritesProvider`. Chaque écran peut consommer l'état via `Provider.of<T>(context)` ou `context.watch<T>()`.

1.3.2. Cycle de vie de la session

1. Au démarrage, `SplashScreen` appelle `AuthProvider.tryAutoLogin()` et `FavoritesProvider.load()` en parallèle.

2. Si un utilisateur est déjà stocké dans `SharedPreferences`, on navigue directement vers l'écran des produits.
3. Sinon, on affiche l'écran de connexion.
4. La déconnexion vide les préférences et ramène à l'écran de connexion.

2. Difficultés rencontrées

2.1. 1. Communication réseau entre le serveur et l'émulateur

La principale difficulté a été la configuration réseau. Sous Android, l'émulateur ne peut pas joindre `localhost` du serveur hôte. Il faut utiliser l'adresse `10.0.2.2` qui redirige vers l'hôte. Pour un appareil réel, il faut utiliser l'IP locale de la machine (`192.168.x.x`). Une URL configurable dans `api_service.dart` a résolu le problème.

2.2. 2. Gestion des états asynchrones

L'enchaînement des appels réseau (connexion → chargement des produits) a nécessité une gestion fine des états de chargement et d'erreur. Le provider notifie les écrans à chaque changement d'état, ce qui a demandé une bonne synchronisation entre les futures et le `notifyListeners()`.

2.3. 3. Persistance des favoris

Le choix de stocker les favoris dans `SharedPreferences` sous forme de chaîne JSON (liste d'IDs) a posé un problème de désérialisation : il faut convertir la liste d'entiers en `Set<int>` au chargement, et inversement au sauvegarde. La gestion en mémoire via un `Set` évite les doublons.

2.4. 4. Filtrage côté serveur

L'implémentation des filtres (recherche textuelle et catégorie) a été faite côté serveur pour alléger le client. La logique de filtrage combine les deux critères avec un `AND`, ce qui a demandé quelques ajustements pour les cas aux limites (caractères accentués, espaces, etc.).

3. Limites de sécurité identifiées

Limite	Impact
Mots de passe en clair	Les mots de passe sont stockés sans hachage dans <code>users.json</code> . Une fuite du fichier expose tous les comptes.
Absence de token JWT	L'authentification repose uniquement sur le stockage local (<code>SharedPreferences</code>). Aucun jeton côté serveur ne permet de valider l'identité au-delà de la connexion initiale.

Limite	Impact
Pas de HTTPS	Les échanges entre l'application et le serveur transitent en HTTP non chiffré. Sur un réseau local non sécurisé, un tiers peut intercepter les identifiants.
CORS ouvert	Le serveur accepte les requêtes de n'importe quelle origine (CORS(app)), ce qui est acceptable en développement mais dangereux en production.
Aucune validation de mot de passe	Ni côté client ni côté serveur, il n'y a de règle de complexité ou de longueur minimale. Le champ est libre.

4. Conclusion

L'architecture choisie (client-serveur REST) est adaptée à ce mini-projet. Elle permet une séparation claire des responsabilités et facilite les tests. Les principales difficultés sont liées à la communication réseau et à la gestion d'état asynchrone en Flutter, tandis que les failles de sécurité sont volontairement présentes dans un cadre pédagogique — elles seraient à corriger dans un contexte professionnel (hachage des mots de passe, JWT, HTTPS).

5. Auteur

Projet réalisé dans le cadre du cours **ITIS - INGETIS - BA DEV/DATA 3**.